Appello 25 settembre 2015

Politecnico di Milano

Anno accademico 2014-2015

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:

Esercizio 1

Si consideri la seguente specifica di un tipo di dato astratto `gara` che rappresenta una gara di atletica leggera (salti e lanci). Alla gara partecipano un numero di atleti fissato nel momento in cui la gara viene creata. Ad ogni atleta corrisponde un numero di pettorale: $1, 2, \ldots, n$, dove n è il numero dei partecipanti alla gara. L'operazione `gareggia(x,ris)` indica che l'atleta con il numero di pettorale x ha gareggiato ottenendo il risultato ris (per esempio, se si tratta di una gara di salto in alto, ris rappresenta l'altezza di salto in cm). L'operazione `primo()` restituisce il numero di pettorale dell'atleta in testa al momento in cui l'operazione viene eseguita.

```
public class Gara{

   // se p <=0 genera l'eccezione NonEsisteGara, altrimenti crea
   //una gara con p partecipanti
   public Gara(int p) throws NonEsisteGaraException

//metodo mutator
   //se x<0 o x  e' maggiore del num. dei partecipanti genera l'eccezione
   //AtletaInesistente altrimenti modifica il valore del risultato
   //ottenuto dall'atleta x senza modificare i risultati degli
   //altri atleti
   public void gareggia(int x, float result) throws AtletaInesistenteException

//metodi observer:

   //restituisce il vincitore corrente della gara.
   public /*@ pure @*/   int primo()

   //restituisce i risultati ottenuti da ogni atleta partecipante alla gara.
   //Se un atleta non ha ancora ancora gareggiato il suo risultato
   //e' convenzionalmente fissato a -1.
   public /*@ pure @*/  ArrayList<Float> risultati;b

}
```

1. Si specifichino i metodi gareggia e primo in JML.

   SOLUZIONE:

   Per completezza specifichiamo anche il costruttore.

```
//@ensures p>0 && risultati().size()==p &&
//@        (\forall int i; 0<= i && i<p; risultati().get(i)==-1);
public Gara(int p) throws NonEsisteGaraException

//@ensures 0<x && x<= risultati().size() &&
//@ risultati().size()== \old(risultati().size() &&
//@ (\forall int i; 0<= i && i<risultati().size();
//@     i==x-1 ? risultati().get(i) == result
//@            :risultati().get(i) == \old(risultati().get(i));
//@signals (AtletaInesistenteException e) (x<=0 || x>risultati().size()) &&
//@         risultati().equals(\old(risultati().clone()));
public void gareggia(int x, float result) throws AtletaInesistenteException

//@ensures risultati().get(\result -1) ==
//@        (\max int i; 0<=i && i<risultati().size(); risultati().get(i));
```

```
public /*@ pure @*/   int primo()
```

Si e' ipotizzato che primo() un pettorale qualunque quando sia chiamato prima di avere inserito almeno un risultato.

2. Si definisca un'implementazione ragionevolmente efficiente per il calcolo del vincitore. Si scrivano pertanto una rep, il suo invariante di rappresentazione e la sua funzione di astrazione.

   SOLUZIONE:

   Per semplificare il calcolo del vincitore e' utile memorizzare il pettorale dell'atleta col migliore risultato. I risultati possono essere memorizzati semplicemente in un array (la cui dimensione non cambia dopo la costruzione). Occorre poi fare attenzione che i pettorali partono da 1 mentre gli indici dell'array da 0.

```
private int primo;
private float[] ris;

//RI:
private invariant ris !=null &&
   ris[primo-1] == (\max int i; 0<=i && i<ris.length; ris[i]);

//AF:
private invariant primo()== primo && ris.equals(risultati().toArray())
```

3. Si fornisca un'implementazione del costruttore e del metodo `gareggia()` descrivendo brevemente, per ciascuno di essi, (1) perchè il rep invariant viene effettivamente mantenuto e (2) perchè la loro specifica viene rispettata. SOLUZIONE:

```
public Gara(int p) throws NonEsisteGaraException {
  if (p<=0) throw new NonEsisteGaraException();
  ris = new Float[p-1];
  primo=1;
  Arrays.fill(ris,-1);
}
```

   Il RI èbanalmente vero (ogni elemento dell'array ris èinizializzato a -1 e il primo e' il pettorale numero 1, che e' un pettorale valido). la specifica e' pure banalmente vera: in base alla funzione di astrazione ris corrisponde esattamente a risultati(), e ris ha dimensione p ed e' inizializzato interamente a -1.

```
public void gareggia(int x, float result) throws AtletaInesistenteException {
  if (x<=0 || x>= ris.length) throw new AtletaInesistenteException();
  ris[x-1] = result;
  if (result>ris[primo-1])
     primo = x;
}
```

   Se RI vale all'entrata del metodo allora ris[primo-1] contiene il risultato migliore (ossia massimo). Se result e' maggiore del massimo corrente allora il codice garantisce di memorizzare in primo il pettorale x. La specifica vale in quanto il codice memorizzare in ris[x-1] il valore di result e, per la funzione di astrazione, ris[x-1] corrisponde a risultati().get(x-1); tutti gli altri valo9re restano invariati.

4. Si vuole ora trattare un caso leggermente più generale di gara: il vincitore può essere scelto anche come l'atleta che ha il punteggio minimo invece che quello con il punteggio massimo (ad esempio quando il punteggio rappresenta il tempo impiegato per una gara di corsa). Si definisca quindi una classe `GaraEstesa`, che estende la classe `Gara`. In essa il costruttore riceve un parametro addizionale `tipo`, il cui valore può essere solo il carattere '<' o '>', per stabilire l'ordinamento, che non può essere modificato dopo la costruzione.

```
public class GaraEstesa extends Gara {



  public GaraEstesa(int p, char tipo)



   //restituisce il vincitore corrente della gara secondo l'ordinamento.
    /@override
   public /*@ pure @*/   int primo()




}
```

a) Si completi la specifica dei metodi di `GaraEstesa`, aggiungendo eventualmente tutti e soli i metodi che si ritengono necessari.

SOLUZIONE:

E' necessario aggiungere un metodo puro che restituisca il tipo della gara:

```
public /*@ pure @*/ char tipo();
```

Specifiche (sotto l'ipotesi che i risultati validi siano positivi).

```
//@requires tipo =='<' || tipo =='>';
//@ensures p>0 && risultati().size()==p &&
//@        tipo()==tipo &&
//@        (\forall int i; 0<= i && i<p; risultati().get(i)==-1);
public GaraEstesa(int p, char tipo) throws NonEsisteGaraException;

//@ensures risultati().get(\result -1) ==
//@ (tipo()=='>' ? (\max int i; 0<=i && i<risultati().size(); risultati().get(i))
//@ : (\min int i; 0<=i&&i<risultati().size()&&risultati().get(i)>0; risultati().get(i));
public /*@ pure @*/   int primo()
```

b) La classe `GaraEstesa` verifica il principio di sostituzione? Giustificare la risposta.

SOLUZIONE:

NO, in quanto il metodo primo() viola la regola dei metodi: quando il tipo e' `'<'`, restituisce il concorrente arrivato ultimo, anziche' il primo. La postcondizione e' pertanto violata.

Esercizio 2

Un impianto industriale coordinata tre bracci meccanici per il confezionamento di caramelle:

- Ogni robot preleva le caramelle da confezionare da un unico contenitore;
- Il contenitore è riempito ad intervalli regolari con 10.000 caramelle;
- Ogni robot può solo prelevare 5 caramelle per volta dal contenitore comune;
- Ogni robot può gestire confezioni da 5, 10 e 15 caramelle;
- L'accesso al contenitore è consentito solamente a due robot per volta;

Si scriva il codice Java delle classi `Robot`, `Controllore` e `Impianto` tenendo presente che: (a) il sistema deve essere progettato per la massima flessibilità, ovvero in futuro il numero di robot e di contenitori della caramelle potrebbero cambiare. Inoltre, in un certo istante, tutti i robot devono gestire confezioni dello stesso tipo. Ad esempio, se un robot è configurato per confezioni da 10 caramelle, anche gli altri devono confezionare allo stesso modo.

Esercizio 3

Si consideri il seguente metodo, in cui ogni istruzione è numerata per comodità:

```
1  static void esegui(int a, int b) {
2                    if (b>= 0 && a>0) {
3                       while (b !=0) {
4                          if (b >= 3)
5                             a++;
6                          else a--;
7                          b--;
                        };
                    };
8                   return a;
   }
```

1. Si scriva la path condition e si sintetizzi un dato di test per percorrere il cammino seguente: 1, 2, 3, 4, 5, 7, 3, 4, 6, 7, 3, 4, 6, 7, 8. SOLUZIONE:

   `b=3 && a>0 (es. b=3 && a = 1)`

2. Qual'è il numero minimo di dati di test necessari per coprire tutte le condizioni del programma? SOLUZIONE:

   Il caso trovato nel punto precedente copre gia' le condizioni del while e del secondo if (durante le varie iterazioni del ciclo). Basta quindi coprire il caso false del primo if (con due casi di test). i casi di test minimi sono quindi solo 3:

   1) b<0 (es- b=-1)
   2) b>=0 && a<=0 (es. b=1 && a=0)
   3) b=3 && a=1

3. Si sintetizzino i dati di test che soddisfano il requisito di cui al punto (2).

Esercizio 4

Si considerino le seguenti classi Java:

```
public class Albero {
   public String toString(){
      return "Albero";
   }
   public void innesto(Albero a) {
      System.out.println("Albero: " + this + " " + a);
   }
}

public class Arancio extends Albero {
   public String toString(){
      return "Arancio";
   }

   public void innesto(Arancio a){
      System.out.println("Arancio: " + this + " " + a);
   }
}

public class Melo extends Albero {
   public String toString(){
      return "Melo";
   }

   public void innesto(Melo m) {
      System.out.println("Melo: " + this + " " + m);
   }
}
```

e si spieghi cosa stamperebbe il seguente metodo main, motivando brevemente le risposte:

```
public class Test {
   public static void main(String args[]) {
      Albero a1, a2;
      Melo m;
      Arancio a;

      a1 = new Albero();
      a2 = new Arancio();
      m = new Melo();
      a = new Arancio();

      m.innesto(a1);
      a2.innesto(a1);
      m.innesto(m);
      a2 = m;
      a1 = a;
      a2.innesto(m);
      a1.innesto(a);
}
```

SOLUZIONE:

Albero: Melo Albero
Albero: Arancio Albero
Melo: Melo Melo
Albero: Melo Melo
Albero: Arancio Arancio